

Branch and Bound Exercises 12.2

Questions and Answers

Question 1

What data structure would you use to keep track of live nodes in a best-first branch-and-bound algorithm?

Answer 1

Use a **priority queue**, also called a **heap**.

Reason:

- In best-first branch-and-bound, live nodes are selected according to their promise.
- The promise of a node is measured by its **bound**.
- Therefore, each live node should be stored with its bound as the priority key.

Selection rule:

problem type	priority	node selected first
minimization	lower bound	smallest lower bound
maximization	upper bound	largest upper bound

Example: If a minimization problem has live nodes with bounds

15, 9, 20,

then the node with bound 9 is expanded first.

Use a priority queue of live nodes, keyed by the node's bound.

Question 2

Solve the following assignment problem using the best-first branch-and-bound algorithm. Use a bounding function based on **matrix rows**.

		job 1	job 2	job 3	job 4
$C =$	person a	9	2	7	8
	person b	6	4	3	7
	person c	5	8	1	8
	person d	7	6	9	4

Answer 2

Row-based lower bound:

For any partial assignment,

$$LB = \text{fixed cost} + \sum_{\text{remaining person } i} \min_{\text{available job } j} c_{ij}.$$

Step 1: Root bound

Find the minimum value in each row:

person	row values	minimum
a	9, 2, 7, 8	2
b	6, 4, 3, 7	3
c	5, 8, 1, 8	1
d	7, 6, 9, 4	4

So the root lower bound is

$$2 + 3 + 1 + 4 = 10.$$

Step 2: Branch on person a

choice	bound calculation	bound
$a \rightarrow 1$	$9 + \min(4, 3, 7) + \min(8, 1, 8) + \min(6, 9, 4)$	17
$a \rightarrow 2$	$2 + \min(6, 3, 7) + \min(5, 1, 8) + \min(7, 9, 4)$	10
$a \rightarrow 3$	$7 + \min(6, 4, 7) + \min(5, 8, 8) + \min(7, 6, 4)$	20
$a \rightarrow 4$	$8 + \min(6, 4, 3) + \min(5, 8, 1) + \min(7, 6, 9)$	18

Best live node after Step 2 is $a \rightarrow 2$ with bound 10.

Step 3: Branch from $a \rightarrow 2$

choice	bound calculation	bound
$a \rightarrow 2, b \rightarrow 1$	$2 + 6 + \min(8, 1) + \min(9, 4)$	13
$a \rightarrow 2, b \rightarrow 3$	$2 + 3 + \min(5, 8) + \min(7, 4)$	14
$a \rightarrow 2, b \rightarrow 4$	$2 + 7 + \min(5, 1) + \min(7, 9)$	17

Best live node after Step 3 is $a \rightarrow 2, b \rightarrow 1$ with bound 13.

Step 4: Complete the best branch

From $a \rightarrow 2, b \rightarrow 1$, the remaining persons are c, d , and the remaining jobs are 3, 4.

completion	total cost	result
$c \rightarrow 3, d \rightarrow 4$	$2 + 6 + 1 + 4 = 13$	best
$c \rightarrow 4, d \rightarrow 3$	$2 + 6 + 8 + 9 = 25$	not best

Optimal assignment:

$$a \rightarrow 2, \quad b \rightarrow 1, \quad c \rightarrow 3, \quad d \rightarrow 4,$$

with total cost

$$2 + 6 + 1 + 4 = 13.$$

Optimal assignment: $(a, 2), (b, 1), (c, 3), (d, 4)$ with cost 13.

Question 3

Solve the following assignment problem using the best-first branch-and-bound algorithm. Use a bounding function based on **matrix columns** rather than rows.

	job 1	job 2	job 3	job 4
person a	9	2	7	8
person b	6	4	3	7
person c	5	8	1	8
person d	7	6	9	4

Answer 3

Column-based lower bound:

For any partial assignment,

$$LB = \text{fixed cost} + \sum_{\text{remaining job } j} \min_{\text{unused person } i} c_{ij}.$$

Step 1: Root bound

Find the minimum value in each column:

job	column values	minimum
1	9, 6, 5, 7	5
2	2, 4, 8, 6	2
3	7, 3, 1, 9	1
4	8, 7, 8, 4	4

So the root lower bound is

$$5 + 2 + 1 + 4 = 12.$$

Step 2: Branch on person a

choice	bound calculation	bound
$a \rightarrow 1$	$9 + \min(4, 8, 6) + \min(3, 1, 9) + \min(7, 8, 4)$	18
$a \rightarrow 2$	$2 + \min(6, 5, 7) + \min(3, 1, 9) + \min(7, 8, 4)$	12
$a \rightarrow 3$	$7 + \min(6, 5, 7) + \min(4, 8, 6) + \min(7, 8, 4)$	20
$a \rightarrow 4$	$8 + \min(6, 5, 7) + \min(4, 8, 6) + \min(3, 1, 9)$	18

Best live node after Step 2 is $a \rightarrow 2$ with bound 12.

Step 3: Branch from $a \rightarrow 2$

choice	bound calculation	bound
$a \rightarrow 2, b \rightarrow 1$	$2 + 6 + \min(1, 9) + \min(8, 4)$	13
$a \rightarrow 2, b \rightarrow 3$	$2 + 3 + \min(5, 7) + \min(8, 4)$	14
$a \rightarrow 2, b \rightarrow 4$	$2 + 7 + \min(5, 7) + \min(1, 9)$	15

Best live node after Step 3 is $a \rightarrow 2, b \rightarrow 1$ with bound 13.

Step 4: Complete the best branch

From $a \rightarrow 2, b \rightarrow 1$, the remaining persons are c, d , and the remaining jobs are 3, 4.

completion	total cost	result
$c \rightarrow 3, d \rightarrow 4$	$2 + 6 + 1 + 4 = 13$	best
$c \rightarrow 4, d \rightarrow 3$	$2 + 6 + 8 + 9 = 25$	not best

Optimal assignment:

$$a \rightarrow 2, \quad b \rightarrow 1, \quad c \rightarrow 3, \quad d \rightarrow 4,$$

with total cost

$$2 + 6 + 1 + 4 = 13.$$

Optimal assignment: $(a, 2), (b, 1), (c, 3), (d, 4)$ with cost 13.

Question 4

Give an example of a best-case input for the branch-and-bound algorithm for the assignment problem. Explain why it is a best-case input.

Answer 4

Idea:

A best-case input is one where the optimal assignment becomes clear very early from the bounds. In such an input, one branch has a much better bound than all other branches.

General example:

For an $n \times n$ minimization assignment problem, take the cost matrix

$$c_{ij} = \begin{cases} 0, & i = j, \\ M, & i \neq j, \end{cases} \quad M \text{ is very large.}$$

Concrete example:

$$C = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & M & M & M \\ 2 & M & 0 & M & M \\ 3 & M & M & 0 & M \\ 4 & M & M & M & 0 \end{array}$$

Optimal assignment:

$$1 \rightarrow 1, \quad 2 \rightarrow 2, \quad \dots, \quad n \rightarrow n$$

This assignment has total cost

$$0 + 0 + \dots + 0 = 0.$$

Why this is best-case:

- The diagonal choices have the smallest possible cost.
- Any off-diagonal choice adds a very large cost M .
- Therefore, the bound of the diagonal branch is much better than the bounds of the other branches.
- Branch-and-bound can prune many non-promising branches quickly.

A best-case input has one clearly dominant low-cost assignment path.

Question 5

In the best case, how many nodes are generated in the state-space tree of the branch-and-bound algorithm for the assignment problem with n persons and n jobs?

Answer 5

Answer:

In the best case, the number of generated nodes is

$$1 + \frac{n(n+1)}{2}.$$

Why:

- The root node is generated first.
- At level 1, there are n possible jobs for the first person.
- In the best case, only one child is promising, so only that child is expanded.
- From that child, there are $n - 1$ possible jobs left.
- Then $n - 2$, then $n - 3$, and so on.

Level-wise count:

stage	nodes generated
root	1
after assigning person 1	n
after assigning person 2	$n - 1$
after assigning person 3	$n - 2$
$\vdots$	$\vdots$
after assigning person n	1

Total:

$$\begin{aligned}\text{nodes} &= 1 + n + (n - 1) + (n - 2) + \cdots + 1 \\ &= 1 + (n + (n - 1) + (n - 2) + \cdots + 1) \\ &= 1 + \frac{n(n+1)}{2}.\end{aligned}$$

Note:

If the question counts only expanded nodes, the best-case count is $n + 1$, because only the root and one node at each assignment level are expanded.

Question 6

Solve the following instance of the knapsack problem by the branch-and-bound algorithm:

item	weight	value	$W = 16.$
1	10	100	
2	7	63	
3	8	56	
4	4	12	

Answer 6

Given:

$$W = 16$$

item	w_i	v_i	v_i/w_i
1	10	100	10
2	7	63	9
3	8	56	7
4	4	12	3

The items are already arranged in decreasing order of value density:

1, 2, 3, 4.

Bounding rule:

At a node with current value v , current weight w , and next item ratio r ,

$$UB = v + (W - w)r.$$

Here UB is an upper bound. If UB is not greater than the best value already found, the node is pruned.

Step 1: Root node

$$UB = 0 + (16 - 0)(10) = 160.$$

Step 2: Branch on item 1

decision	items taken	w	v	UB
$x_1 = 1$	$\{1\}$	10	100	$100 + (16 - 10)(9) = 154$
$x_1 = 0$	$\emptyset$	0	0	$0 + (16)(9) = 144$

The best live node is $x_1 = 1$, because $154 > 144$.

Step 3: Branch from $x_1 = 1$

decision	items taken	w	v	result
$x_1 = 1, x_2 = 1$	$\{1, 2\}$	17	163	infeasible
$x_1 = 1, x_2 = 0$	$\{1\}$	10	100	$UB = 100 + (6)(7) = 142$

So the node $x_1 = 1, x_2 = 0$ remains live.

Step 4: Explore promising live nodes

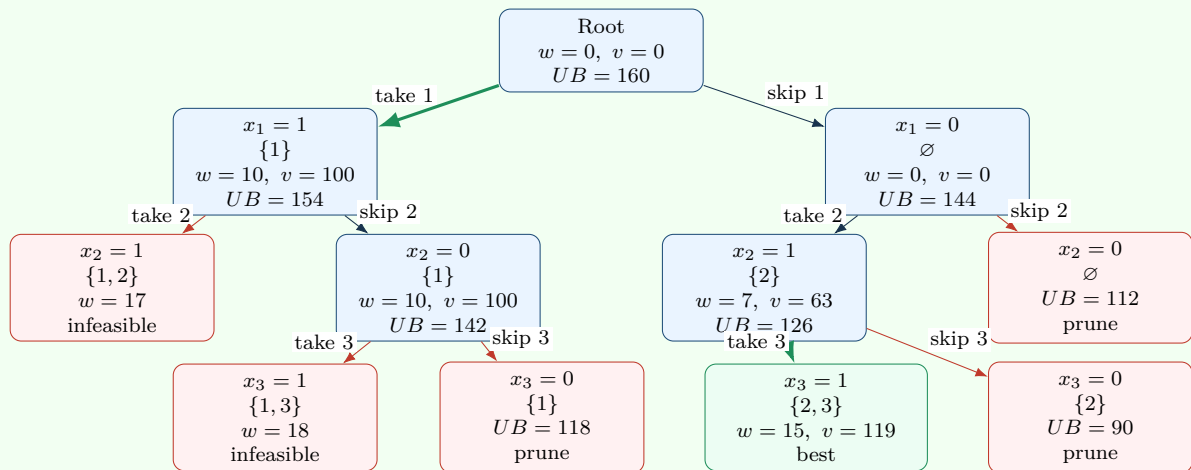
node	items taken	w	v	UB
$x_1 = 0, x_2 = 1$	$\{2\}$	7	63	$63 + (9)(7) = 126$
$x_1 = 0, x_2 = 0$	$\emptyset$	0	0	$0 + (16)(7) = 112$
$x_1 = 1, x_2 = 0, x_3 = 0$	$\{1\}$	10	100	$100 + (6)(3) = 118$

The branch $x_1 = 0, x_2 = 1$ is promising. Continue from it:

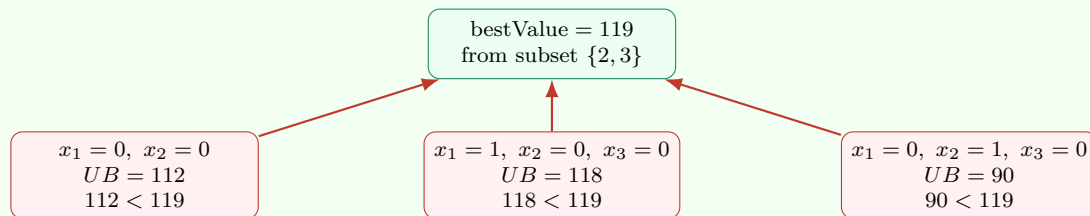
decision	items taken	w	v
$x_1 = 0, x_2 = 1, x_3 = 1$	$\{2, 3\}$	15	119
$x_1 = 0, x_2 = 1, x_3 = 0$	$\{2\}$	7	63

The feasible solution $\{2, 3\}$ gives value 119. Any remaining node with upper bound at most 119 is pruned.

State-space tree:



Pruning comparison after finding best value 119:



Final answer:

items selected = $\{2, 3\}$

total weight = $7 + 8 = 15 \leq 16$

total value = $63 + 56 = 119$

Optimal set = $\{2, 3\}$, weight = 15, value = 119.

Question 7

Suggest a stronger bounding function for solving the 0/1 knapsack problem by branch-and-bound. Explain how the bound is computed and why it is valid.

Answer 7

Answer:

A stronger bounding function is the **fractional knapsack upper bound**.

Basic idea:

- First, sort the remaining items by decreasing value-to-weight ratio v_i/w_i .
- Add full items greedily while they fit in the remaining capacity.
- If the next item does not fit completely, add only the fractional part needed to fill the capacity.
- This fractional value is used only for calculating the bound, not for the final 0/1 solution.

Formula:

Suppose a node has current value v , current weight w , and knapsack capacity W . Then

$$UB = v + \text{best fractional value that can fit in } (W - w).$$

Why it is valid:

- The 0/1 knapsack problem allows either taking an item or not taking it.
- The fractional knapsack relaxation also allows taking part of an item.
- Therefore, the fractional version can never give a smaller maximum value than the 0/1 version.
- So its value is a valid upper bound for all solutions below that node.

Pruning rule:

If

$$UB \leq \text{current best value},$$

then the node is pruned because it cannot lead to a better solution.

Use the fractional knapsack relaxation as the bounding function.

Question 8

Use the fractional-knapsack upper bound from the previous question in the branch-and-bound algorithm for the knapsack instance of Question 6.

Answer 8**Given instance:**

item	w_i	v_i	v_i/w_i	
1	10	100	10	$W = 16.$
2	7	63	9	
3	8	56	7	
4	4	12	3	

The items are already sorted by decreasing value-to-weight ratio.

How the Fractional Part Is Calculated

For the upper bound, we temporarily allow a fraction of the next item. If the remaining capacity is R and the next item has weight w_i , then the fraction used is

$$\frac{R}{w_i}.$$

The value contributed by that fraction is

$$\frac{R}{w_i} \cdot v_i.$$

This is only for estimating UB . The final knapsack solution must still use whole items only.

Step 1: Root upper bound

At the root, the remaining capacity is 16. Fill it greedily by ratio:

action	weight used	value gained	remaining capacity
take item 1 fully	10	100	$16 - 10 = 6$
take $6/7$ of item 2	6	$\frac{6}{7} \cdot 63 = 54$	0

So,

$$100 + \frac{6}{7} \cdot 63 = 100 + 54 = 154.$$

$$UB(\text{root}) = 154.$$

Step 2: Branch on item 1

If $x_1 = 1$, item 1 is already selected. Its weight is 10, so the remaining capacity is

$$16 - 10 = 6.$$

The next best item is item 2, whose weight is 7. Since only 6 units of capacity remain, the fractional bound uses

$$\frac{6}{7}$$

of item 2.

If $x_1 = 0$, item 1 is excluded. We fill capacity using the next best items:

$$\text{item 2 fully, item 3 fully, } \frac{1}{4} \text{ of item 4.}$$

The fraction $\frac{1}{4}$ appears because after items 2 and 3, the remaining capacity is

$$16 - 7 - 8 = 1,$$

and item 4 has weight 4, so the fraction is $1/4$.

node	w	v	UB
$x_1 = 1$	10	100	$100 + \frac{6}{7} \cdot 63 = 154$
$x_1 = 0$	0	0	$63 + 56 + \frac{1}{4} \cdot 12 = 122$

Best live node: $x_1 = 1$ because $154 > 122$.

Step 3: Branch from $x_1 = 1$

For node $x_1 = 1, x_2 = 0$, item 1 is selected and item 2 is excluded:

$$w = 10, \quad v = 100.$$

The remaining capacity is

$$16 - 10 = 6.$$

The next available item is item 3, whose weight is 8 and value is 56. Since only 6 units fit, the fractional part is

$$\frac{6}{8}.$$

So its fractional contribution is

$$\frac{6}{8} \cdot 56 = 42.$$

node	w	v	result
$x_1 = 1, x_2 = 1$	17	163	infeasible
$x_1 = 1, x_2 = 0$	10	100	$UB = 100 + \frac{6}{8} \cdot 56 = 142$

The node $x_1 = 1, x_2 = 0$ is still promising because its upper bound is 142.

Step 4: Branch from $x_1 = 1, x_2 = 0$

node	w	v	result
$x_1 = 1, x_2 = 0, x_3 = 1$	18	156	infeasible
$x_1 = 1, x_2 = 0, x_3 = 0, x_4 = 1$	14	112	feasible

So the current best feasible value is

$$\text{best} = 112$$

with items $\{1, 4\}$.

Step 5: Explore the other promising branch $x_1 = 0$

For node $x_1 = 0, x_2 = 1$, item 2 is selected:

$$w = 7, \quad v = 63.$$

The remaining capacity is

$$16 - 7 = 9.$$

Now take item 3 fully:

$$w_3 = 8, \quad v_3 = 56,$$

leaving

$$9 - 8 = 1$$

unit of capacity. Item 4 has weight 4, so only

$$\frac{1}{4}$$

of item 4 is used in the bound.

node	w	v	UB
$x_1 = 0, x_2 = 1$	7	63	$63 + 56 + \frac{1}{4} \cdot 12 = 122$
$x_1 = 0, x_2 = 0$	0	0	$56 + 12 = 68$

The node $x_1 = 0, x_2 = 1$ is promising because $122 > 112$.

Step 6: Branch from $x_1 = 0, x_2 = 1$

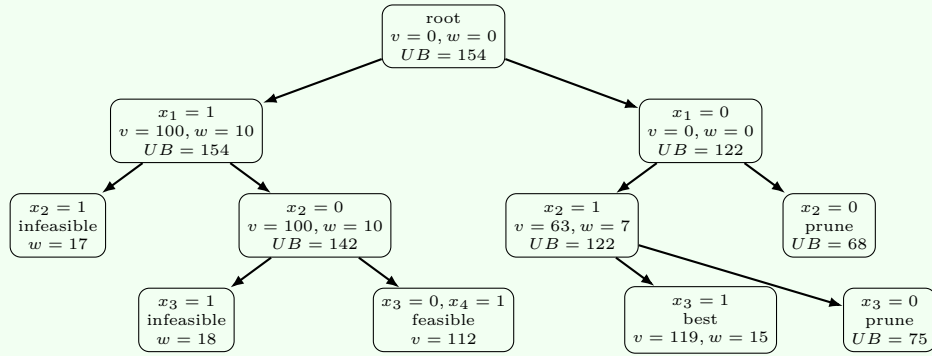
node	w	v	result
$x_1 = 0, x_2 = 1, x_3 = 1$	15	119	feasible
$x_1 = 0, x_2 = 1, x_3 = 0$	7	63	$UB = 63 + 12 = 75$

The feasible solution $\{2, 3\}$ gives

$$v = 63 + 56 = 119, \quad w = 7 + 8 = 15.$$

Now the current best value is 119. All remaining live nodes have upper bound at most 119, so they are pruned.

Tree view:



Final answer:

Optimal set = $\{2, 3\}$, weight = 15, value = 119.

Question 9

For a symmetric TSP distance matrix, prove that the following expression is a valid lower bound on the length of any Hamiltonian tour:

$$lb = \left\lceil \frac{s}{2} \right\rceil,$$

where s is found as follows: for each city, add the two smallest edge lengths incident with that city; then add these values over all cities.

Answer 9

Direct answer:

The bound is valid because every TSP tour uses exactly two edges incident with each city. The two tour edges chosen at a city cannot have smaller total length than the two shortest available edges at that city.

Notation:

Let

$$L = \text{length of any Hamiltonian tour.}$$

For each city i , define

$$s_i = \text{sum of the two shortest edge lengths incident with city } i.$$

Then

$$s = s_1 + s_2 + \cdots + s_n.$$

The proposed lower bound is

$$lb = \left\lceil \frac{s}{2} \right\rceil$$

Why s_i is a lower estimate at city i :

- In a Hamiltonian tour, the tour enters city i once and leaves city i once.
- Therefore, the tour uses exactly two edges incident with city i .
- Among all edges incident with city i , the two shortest have total length s_i .
- So the two tour edges at city i must have total length at least s_i .

Therefore, for every city i ,

$$\text{two tour edges at city } i \geq s_i.$$

Add the inequalities for all cities:

$$s_1 + s_2 + \cdots + s_n \leq 2L,$$

where L is the length of the tour.

Why the right side is $2L$

When we add city-by-city edge costs, every tour edge is counted twice: once from one endpoint and once from the other endpoint. Therefore, the total counted tour cost is $2L$, not L .

So,

$$s \leq 2L.$$

Dividing by 2,

$$\frac{s}{2} \leq L.$$

Since all distances are integers, every tour length L is also an integer. Therefore, if $s/2$ is not an integer, the lower bound can be rounded up:

$$lb = \left\lceil \frac{s}{2} \right\rceil.$$

Thus this is a valid lower bound for the optimal TSP tour length.

Question 10

How would you modify this lower bound for nonsymmetric distance matrices in the traveling salesman problem?

Answer 10

Problem with the symmetric lower bound:

The previous lower bound assumes a symmetric distance matrix, where

$$d_{ij} = d_{ji}.$$

For a nonsymmetric matrix,

$$d_{ij} \neq d_{ji}$$

may be possible. So we must consider direction.

Directed-tour property:

In a directed Hamiltonian tour:

- each city has exactly one outgoing edge,
- each city has exactly one incoming edge.

Outgoing lower bound:

For each city i , choose the smallest outgoing distance:

$$out_i = \min_{j \neq i} d_{ij}.$$

Then

$$LB_{\text{out}} = \sum_i out_i.$$

Incoming lower bound:

For each city j , choose the smallest incoming distance:

$$in_j = \min_{i \neq j} d_{ij}.$$

Then

$$LB_{\text{in}} = \sum_j in_j.$$

Modified lower bound:

Since both LB_{out} and LB_{in} are valid lower bounds, we can take the stronger one:

$$LB = \max \left\{ \sum_i \min_{j \neq i} d_{ij}, \sum_j \min_{i \neq j} d_{ij} \right\}.$$

Why this works:

- Every tour must leave each city once, so the sum of cheapest outgoing edges is a lower bound.
- Every tour must enter each city once, so the sum of cheapest incoming edges is also a lower bound.
- Taking the maximum gives the better of these two valid lower bounds.

If a direct average-style version is required, another valid but weaker bound is

$$\left\lceil \frac{1}{2} \left(\sum_i \min_{j \neq i} d_{ij} + \sum_j \min_{i \neq j} d_{ij} \right) \right\rceil.$$

Question 11

Apply the branch-and-bound algorithm to solve the traveling salesman problem for the following graph:

$$ab = 2, \quad ac = 5, \quad ad = 7, \quad bc = 8, \quad bd = 3, \quad cd = 1.$$

Answer 11

Goal:

Find the shortest Hamiltonian circuit.

Graph:

There are four vertices:

$$\{a, b, c, d\}.$$

The edge lengths are:

edge	ab	ac	ad	bc	bd	cd
length	2	5	7	8	3	1

Lower-bound rule:

For each vertex, take the two shortest incident edges. Let s be the sum of all these values. Then

$$LB = \left\lceil \frac{s}{2} \right\rceil.$$

Why Divide by 2

A Hamiltonian circuit uses exactly two edges incident with each vertex. When we add city-by-city edge costs, every selected tour edge is counted twice, once from each endpoint.

Step 1: Compute the root lower bound

vertex	two smallest incident edges	sum
a	$ab = 2, ac = 5$	7
b	$ab = 2, bd = 3$	5
c	$cd = 1, ac = 5$	6
d	$cd = 1, bd = 3$	4

$$s = 7 + 5 + 6 + 4 = 22.$$

Therefore,

$$LB = \left\lceil \frac{22}{2} \right\rceil = 11.$$

Step 2: Branch from starting vertex a

Fix the starting vertex as a . Since the graph is undirected, reverse tours are duplicates, so we only need to consider one direction consistently.

The first-level branches are:

$$a, b \quad a, c \quad a, d.$$

Branch	Meaning	Bound or status
a, b	tour begins $a \rightarrow b$	$LB = 11$
a, c	reverse-duplicate direction of a tour beginning $a \rightarrow b$	not used
a, d	tour begins $a \rightarrow d$	$LB = 14$

How the branch bounds are obtained:

For a, b , the forced edge is $ab = 2$. This edge is already one of the two cheapest incident edges at both a and b , so the lower bound remains

$$LB(a, b) = 11.$$

For a, d , the forced edge is $ad = 7$. It must be counted at both a and d :

vertex	two edges used in bound	sum
a	$ab = 2, ad = 7$	9
b	$ab = 2, bd = 3$	5
c	$cd = 1, ac = 5$	6
d	$dc = 1, da = 7$	8

$$LB(a, d) = \left\lceil \frac{9 + 5 + 6 + 8}{2} \right\rceil = \left\lceil \frac{28}{2} \right\rceil = 14.$$

So the most promising branch is

$$\boxed{a, b}$$

because

$$11 < 14.$$

Step 3: Complete tours under a, b

After choosing

$$a \rightarrow b,$$

the remaining vertices are

$$\{c, d\}.$$

There are only two possible completions:

Choice after a, b	Complete tour	Length
Go to c next	$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$	$2 + 8 + 1 + 7 = 18$
Go to d next	$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$	$2 + 3 + 1 + 5 = 11$

The second completion gives a feasible tour:

$$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$$

with length

$$11.$$

Step 4: Compare with the lower bound

The root lower bound was

$$LB_{\text{root}} = 11.$$

The tour found has actual length

$$11.$$

Therefore,

$$\text{actual tour length} = \text{lower bound}.$$

Why This Proves Optimality

No tour can have length less than the lower bound 11. Since we found a valid tour with length 11, no better tour is possible.

Pruning consequence:

After finding a tour of length 11, any branch with

$$LB \geq 11$$

cannot improve the answer. In particular,

$$LB(a, d) = 14 \geq 11,$$

so the a, d branch is pruned.

Check all distinct tours:

For four vertices, after fixing a and ignoring reverse duplicates, the distinct tours are:

tour	length
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$	$2 + 8 + 1 + 7 = 18$
$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$	$2 + 3 + 1 + 5 = 11$
$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$	$5 + 8 + 3 + 7 = 23$

The minimum is 11.

Final answer:

Optimal tour: $a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$, length = 11.

Equivalent tour:

The reverse tour

$$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$$

also has length

$$5 + 1 + 3 + 2 = 11.$$

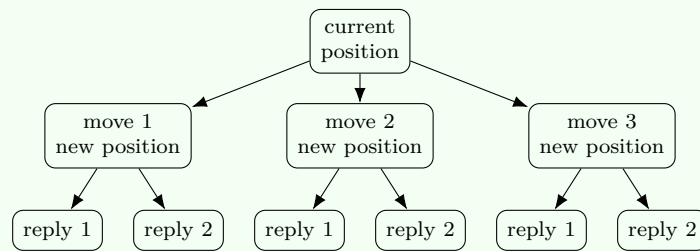
Question 12

Explain how state-space trees are used in game-playing programs such as chess, checkers, and tic-tac-toe. Also explain the role of the minimax algorithm and alpha-beta pruning.

Answer 12**State-space tree in games:**

- Each node represents a game position.
- The root node represents the current position.
- Each edge represents a legal move.
- Each child node represents the new position after making that move.
- A path from the root represents a sequence of moves.

Simple state-space tree:



Examples:

game	node represents	edge represents
tic-tac-toe	board position	placing X or O
checkers	piece arrangement	moving or capturing
chess	chess position	a legal chess move

Minimax algorithm:

The **minimax algorithm** is used for two-player, zero-sum games.

- One player is the **maximizer**.
- The other player is the **minimizer**.
- The maximizer tries to get the largest evaluation score.
- The minimizer tries to get the smallest evaluation score.
- The algorithm assumes both players play optimally.

level in tree	operation
max player's turn	choose the maximum child value
min player's turn	choose the minimum child value

Alpha-beta pruning:

Alpha-beta pruning improves minimax by avoiding branches that cannot affect the final decision.

- α : best value currently guaranteed for the maximizer.
- β : best value currently guaranteed for the minimizer.

If during search,

$$\alpha \geq \beta,$$

then the remaining children of that node are pruned.

Why pruning is useful:

- It gives the same final move as minimax.
- It avoids searching branches that cannot change the decision.
- It reduces the number of nodes examined.
- It makes deeper search possible in large games.

Small vs large games:

game type	search strategy
tic-tac-toe	often possible to search the full tree
chess/checkers	use depth limits and evaluation functions

Game programs use state-space trees to represent possible moves, minimax to choose the best move, and alpha-beta pruning to avoid unnecessary search.

State-space tree + minimax + alpha-beta pruning

Question 13

Write a program for solving the assignment problem by the branch-and-bound algorithm. Experiment with your program to determine the average size of the cost matrices for which the problem is solved in a given amount of time, say, 1 minute on your computer.

Answer 13

A suitable program uses a priority queue of live nodes. Each node stores a partial assignment, its current cost, the set of used columns, and a lower bound.

```
from heapq import heappush, heappop
from math import inf

def lower_bound(cost, row, used_cols, current):
    n = len(cost)
    bound = current
    for r in range(row, n):
        best = min(cost[r][c] for c in range(n) if c not in used_cols)
        bound += best
    return bound

def assignment_branch_bound(cost):
    n = len(cost)
    best_cost = inf
    best_assignment = None
    pq = []

    root_bound = lower_bound(cost, 0, set(), 0)
    heappush(pq, (root_bound, 0, 0, tuple(), frozenset()))

    while pq:
        bound, row, current, assign, used = heappop(pq)
        if bound >= best_cost:
            continue

        if row == n:
            best_cost = current
            best_assignment = assign
            continue

        for col in range(n):
            if col in used:
                continue
            new_cost = current + cost[row][col]
            new_used = set(used)
```

```

        new_used.add(col)
        new_assign = assign + (col,)
        new_bound = lower_bound(cost, row + 1, new_used, new_cost)
        if new_bound < best_cost:
            heappush(
                pq,
                (new_bound, row + 1, new_cost,
                 new_assign, frozenset(new_used))
            )

    return best_cost, best_assignment

```

To estimate the average matrix size solvable in 1 minute:

1. Generate random $n \times n$ cost matrices.
2. Run the algorithm several times for each n .
3. Record the average running time.
4. Increase n until the average time is close to 60 seconds.

The maximum matrix size solvable in 1 minute depends on the computer, programming language, implementation details, and random matrix distribution.

Question 14

Write a program to solve the knapsack problem with the branch-and-bound algorithm.

Answer 14

The following program uses the fractional knapsack bound and a priority queue. It solves a 0/1 knapsack maximization problem.

```

from heapq import heappush, heappop

def fractional_bound(items, capacity, level, weight, value):
    if weight > capacity:
        return 0

    bound = value
    total_weight = weight
    n = len(items)
    i = level

    while i < n and total_weight + items[i][0] <= capacity:
        w, v, original_index = items[i]
        total_weight += w
        bound += v
        i += 1

    if i < n:
        w, v, original_index = items[i]
        bound += (capacity - total_weight) * (v / w)

    return bound

def knapsack_branch_bound(weights, values, capacity):
    n = len(weights)

```

```

items = sorted(
    [(weights[i], values[i], i) for i in range(n)],
    key=lambda x: x[1] / x[0],
    reverse=True
)

best_value = 0
best_taken = set()
pq = []

root_bound = fractional_bound(items, capacity, 0, 0, 0)
heappush(pq, (-root_bound, 0, 0, 0, frozenset()))

while pq:
    neg_bound, level, weight, value, taken = heappop(pq)
    bound = -neg_bound

    if bound <= best_value or level == n:
        continue

    w, v, original_index = items[level]

    # Include the current item.
    new_weight = weight + w
    new_value = value + v
    new_taken = set(taken)
    new_taken.add(original_index)

    if new_weight <= capacity and new_value > best_value:
        best_value = new_value
        best_taken = new_taken

    include_bound = fractional_bound(
        items, capacity, level + 1, new_weight, new_value
    )
    if include_bound > best_value:
        heappush(
            pq,
            (-include_bound, level + 1,
             new_weight, new_value, frozenset(new_taken))
        )

    # Exclude the current item.
    exclude_bound = fractional_bound(
        items, capacity, level + 1, weight, value
    )
    if exclude_bound > best_value:
        heappush(
            pq,
            (-exclude_bound, level + 1,
             weight, value, taken)
        )

return best_value, sorted(best_taken)

```

For the knapsack instance solved earlier, this program returns value 119 and items {2, 3}.